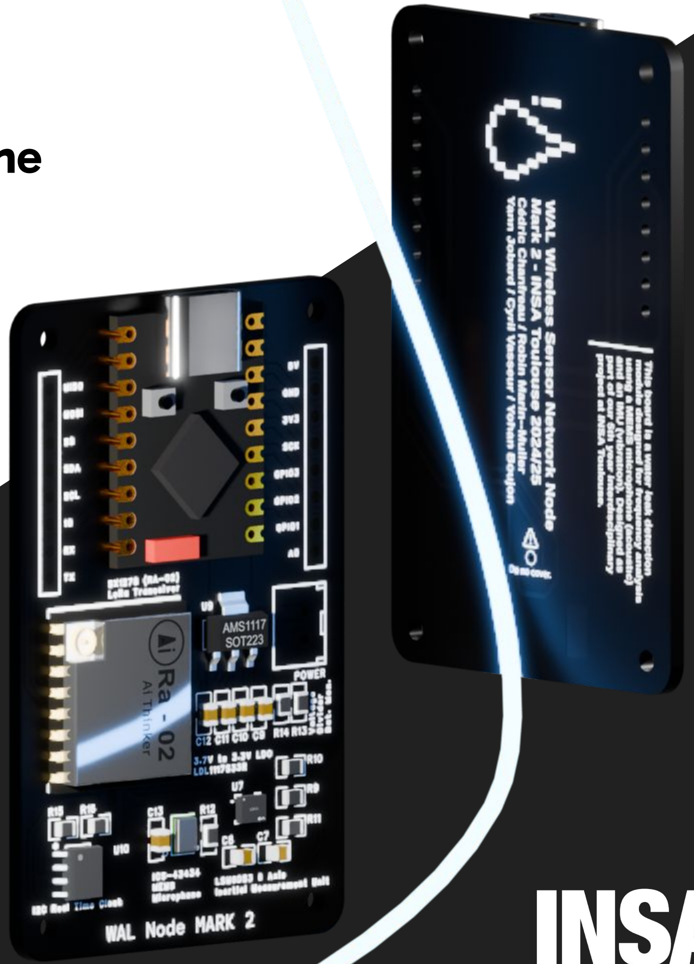
# Non-Intrusive Detection of Water Leaks in Pipeline Networks Using Vibration and Acoustic Sensors

*An Innovative Project*
*PTP Innovative Smart Systems*




Yohan Boujon
Robin Marin--Muller
Cyril Vasseur
Cédric Chanfreau
Yann Jobard

INSA TOULOUSE

1 **Water Leak Detection System**

Detect leak without being intrusive

2 **Sensors and Analysis**

Gathering the data, learning the user’s behavior

3 **Node-to-Node Communication**

Which frequency, securing the data, custom packet

4 **Processing Data on the Server**

Subscribe-publish data, storing it in a database

5 **Displaying the Informations**

Web and mobile application

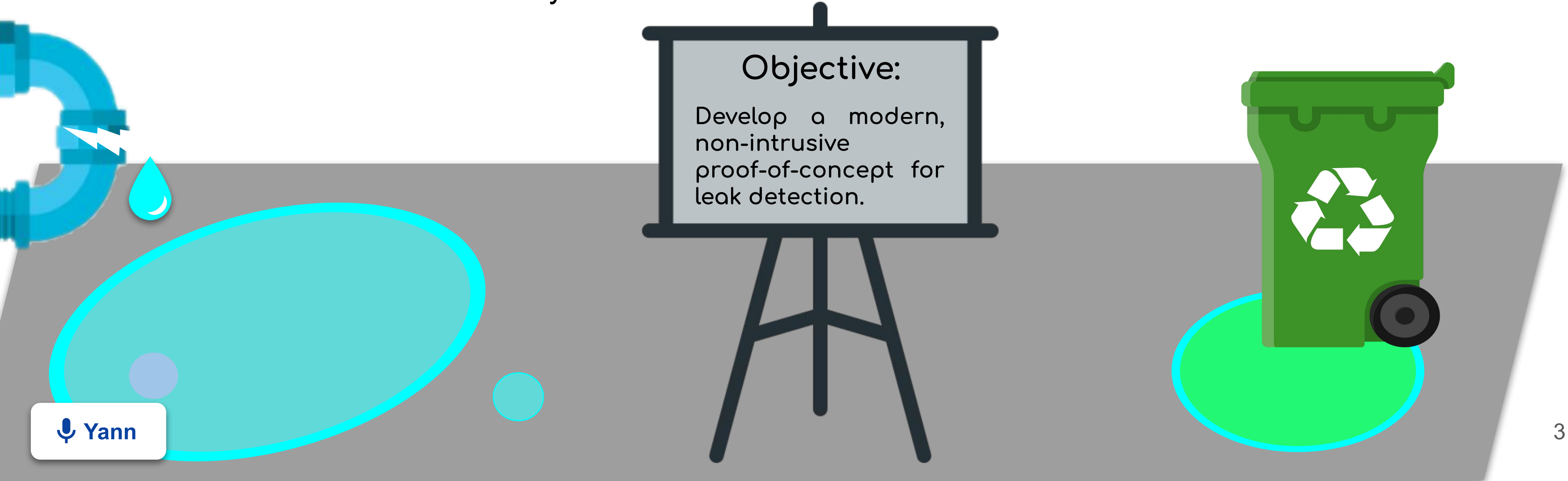
6 **Project Results and Limits**

Consumption, Success and Limitation

**What A Leak!**

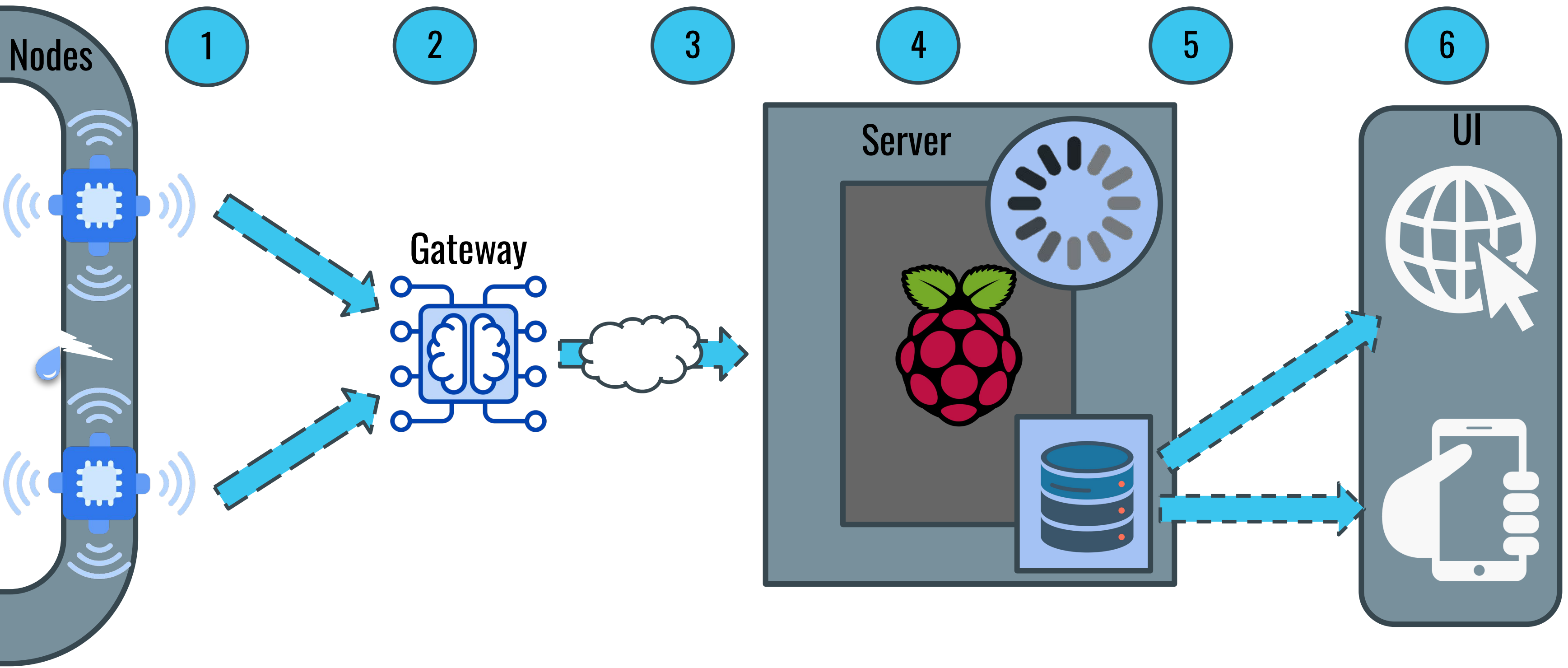
# Problem Statement

- In France, **20%** of the water transported through the distribution network is lost due to leaks. (**1 billion m³**)
- Undetected leaks in domestic pipework can cost up to: **1,000 to 10,000€ /year.**
- For public infrastructure that more than **4 billion euros in France.**
- Traditional methods are costly and inefficient.




Yann

# Overview

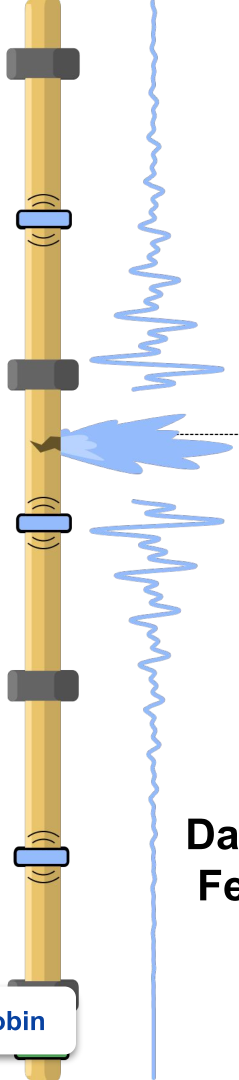



1. N [illegible] vibration data using accelerometers and microphones.
2. [illegible] LoRa to the gateway that use Machine Learning to determine if there is a leak
3. The data is sent to the **Server** through the web
4. The gateway processes the data using machine learning to detect abnormal behavior.
5. Results are stored in a database.
6. User interfaces (Website and App) retrieve relevant information and conclusions from the database.

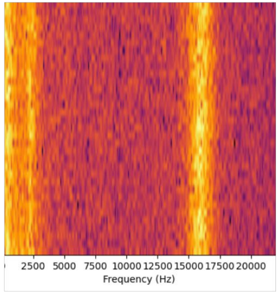
Yann

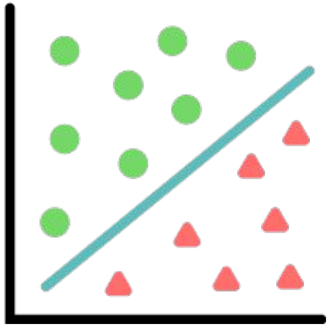
# Principle and Architecture

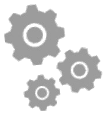
*How can we identify leaks, what principles can we use?*



Leaks emit distinct spectral signatures





⚠ Leak

✓ No Leak

**Data Collection and Feature Extraction**

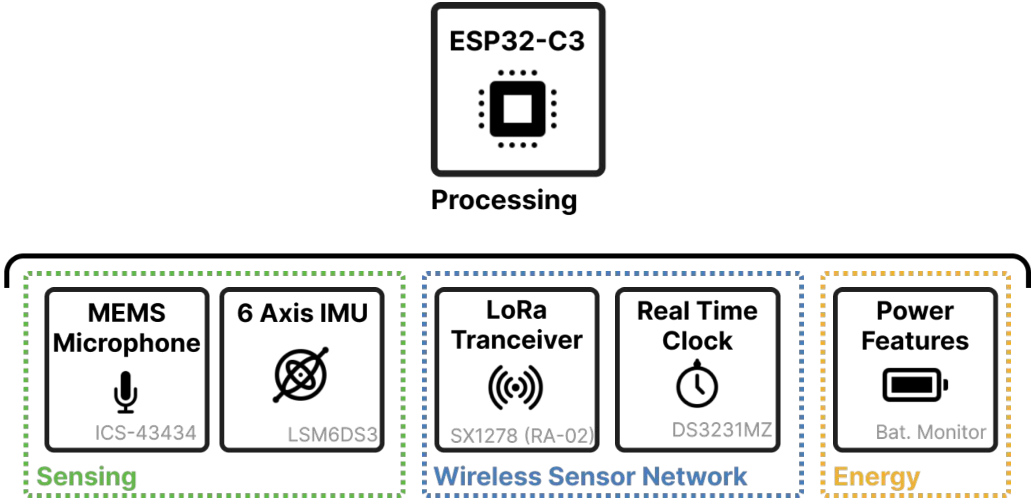
**SVN Machine Learning Classifier**

**Model Prediction**

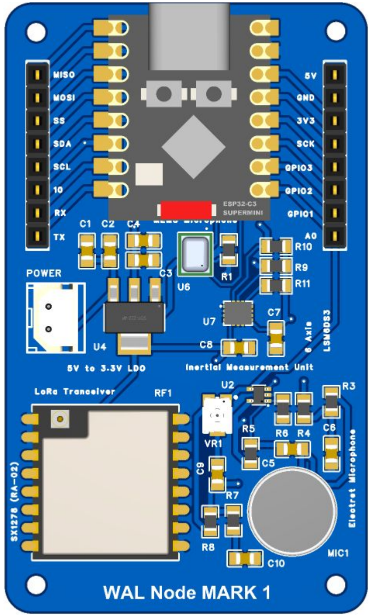
Robin

# Hardware Architecture

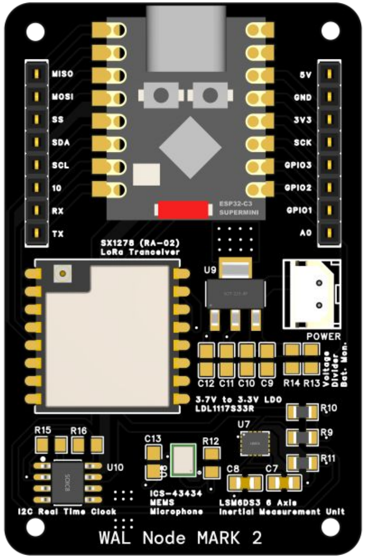
*What hardware is required for effective leak detection?*




Hardware Architecture




PCB Rev 1




PCB Rev 2

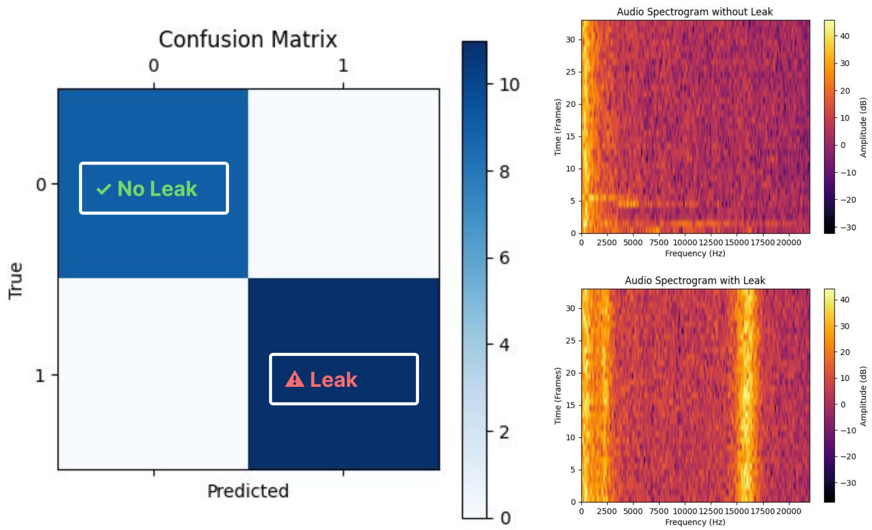
# Machine Learning

*How can machine learning improve leak detection accuracy?*

**Traditional Methods?:** Unsuitable and hard to implement due to unpredictable noise.

**Feature Extraction:** Using Fast Fourier Transform (FFT) to identify frequency patterns unique to leaks.

**Algorithm:** Support Vector Machines (SVM) for classification.



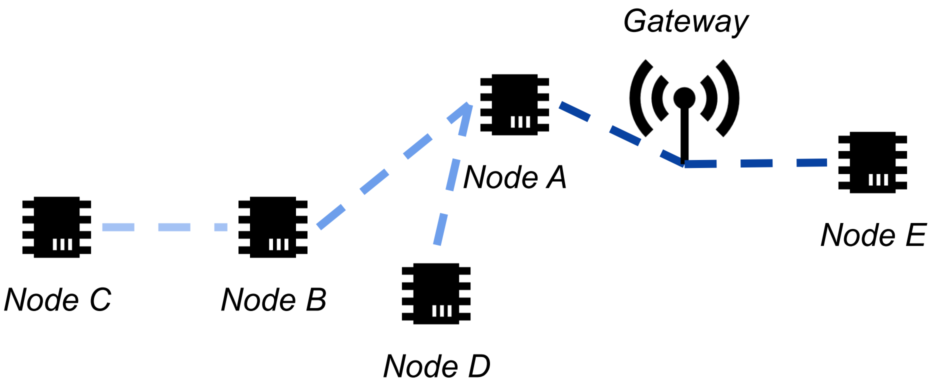

## ML Results

# LoRa communication

- **Choosing the Right Frequency**:
  - LoRa on *433MHz* can go through many different materials
  - *(Possible Upgrade) Lower frequencies* with a custom implementation

- **266m** in an unobstructed area
- **35m** with 2 well-isolated floors

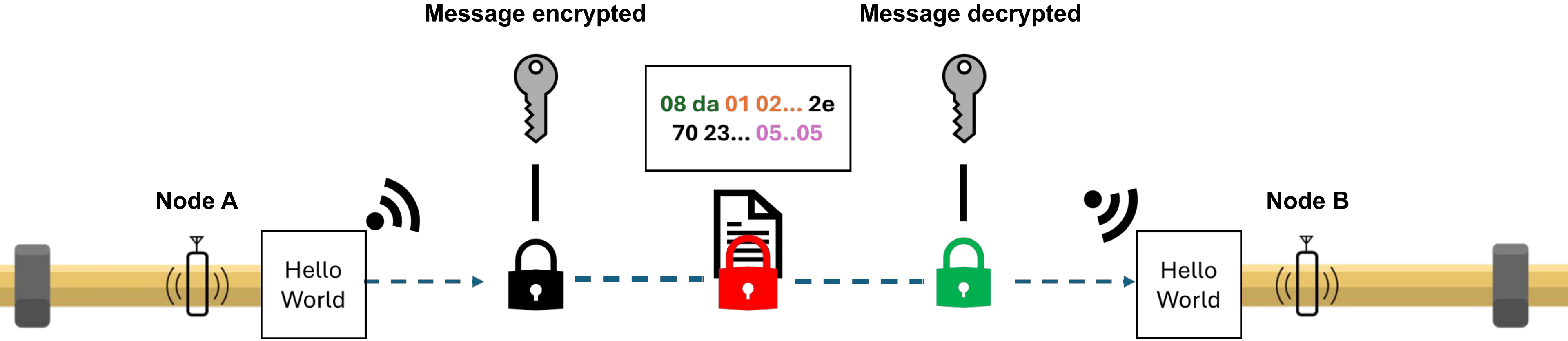
- **Mesh Network Structure**:
  - Each node can communicate with its *neighbor*
  - *Sending* its information until the gateway is reached



# Secure the data

Without security, anyone could intercept, modify or resend the data, leading to false alarm or missing real leaks.

**Same static key between all nodes.**




**IV (Initialisation Vector):**

- Add randomness to encryption (Prevents Replay Attacks).
- Unique encrypted data.
- Minimal overhead (Custom 2 Dynamic 14 Static Bytes Algorithm).
- Enhanced data security.

**Padding :**

- Extra bytes to data.
- Block size alignment (16 Bytes).
- Removed after decryption.

Cedric

# Custom protocol

- **Using the less possible memory**:
  - 256 Bytes *maximum* can be send each time
  - It takes *multiple milliseconds* to send this data each time


**WAL Protocol**

*before topology* *10 bytes*

| | | | |
|---|---|---|---|
| WAL Header *(3 bytes)* | Address ***SRC*** *(3 bytes)* | Address ***DST*** *(3 bytes)* | Command *(1 byte)* |

*after topology* *6 bytes*

| | | | |
|---|---|---|---|
| WAL Header *(3 bytes)* | Node ***SRC*** *(1 byte)* | Node ***DST*** *(1 byte)* | Command *(1 byte)* |

*The destination changes at every node.*

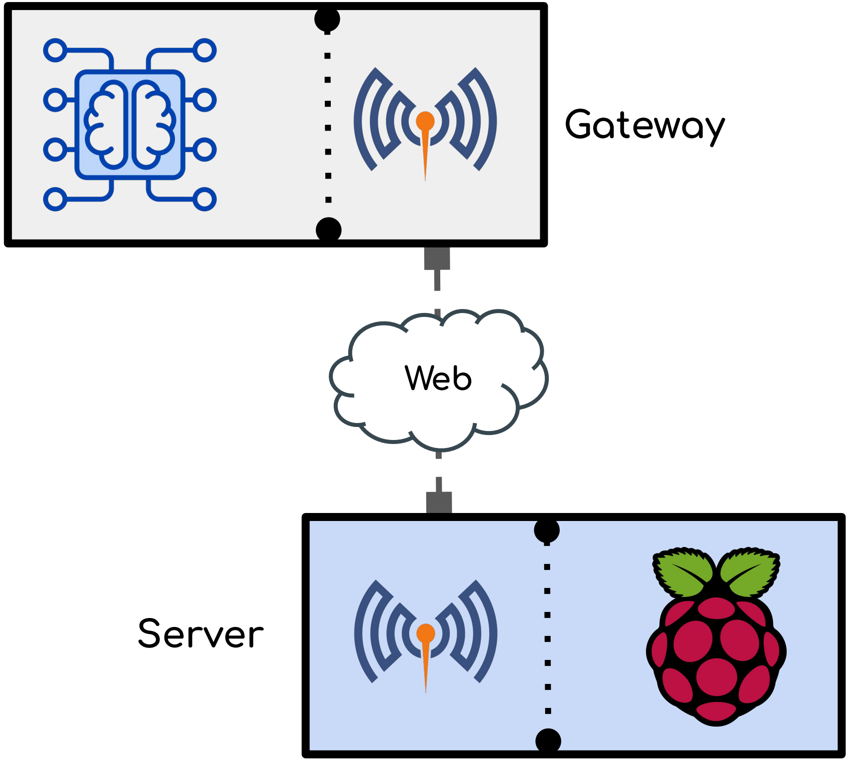
- Topology Payload
- Time Payload
- Data Payload
- Battery Payload

# Custom protocol

- **Interpreting the data**:
  - The data has to be *easily* readable for each operation
  - *MQTT* with a simple publisher can transfer the data to the *internet*

# Server-Side Processing

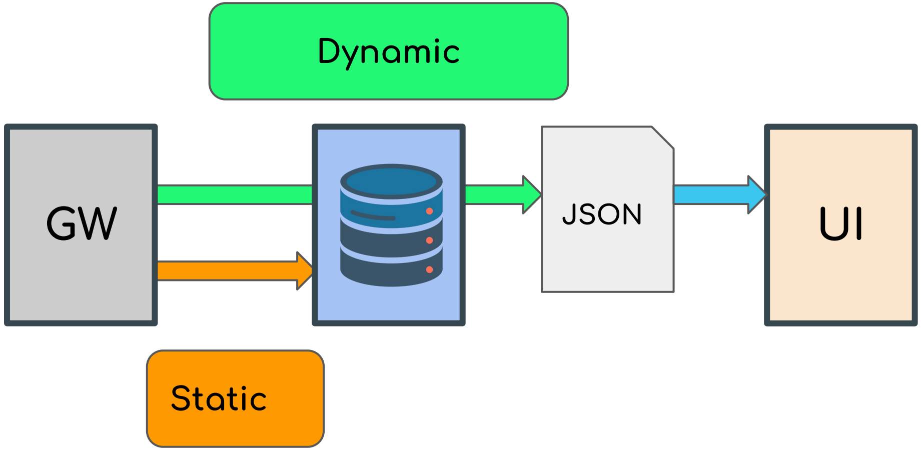
- **Data Reception through web**:
  - MQTT Client Gateway
  - MQTT broker receives sensor data
- **Security:**
  - MQTT communication limited by credentials
  - Firewall configuration
  - On reception, server throws inadequate Data received




Yann

# Server-Side Processing

- **Two SQLite Table:**
  - Dynamic Data: changing <u>often</u>
  - Static Data: changing occasionally
- **Dynamic Data stored:**
  - Node 's ID
  - Measures:
    - FFT
    - Status
    - Timestamp
  - Accessory:
    - Temperature
    - Battery
- **Static Data**:
  - Location
  - UI Personalization
- **Processing**:
  - Format static data into JSON for visualization on web and mobile platforms



# Web interface

**Web Application**:

- Developed with **Spring Boot** and **Maven**
- **Accessible** from everywhere

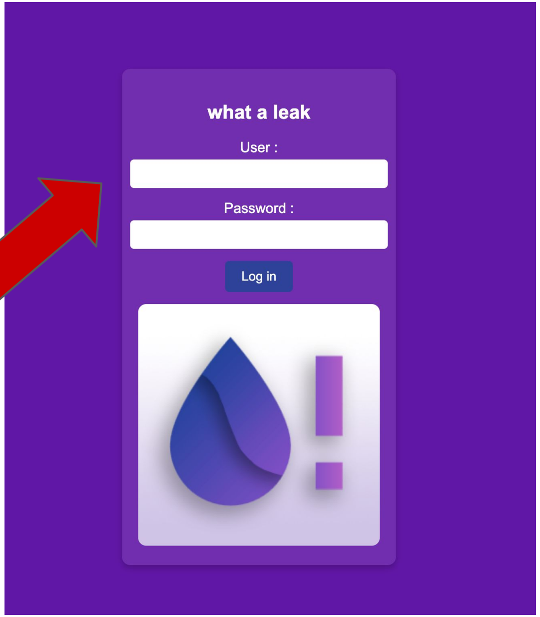
**Main feature:**

- Displays real-time leak alerts, FFT visualizations, and battery levels
- Team

**Real time update:**

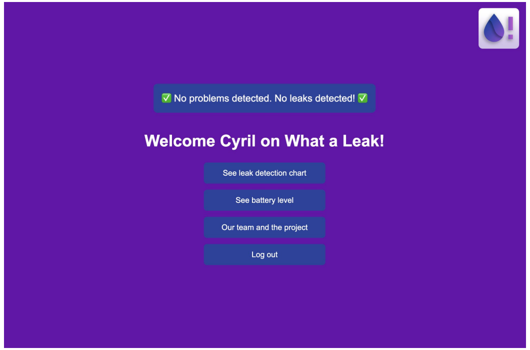
- Each update on the DB

robust against sql injection




User Login

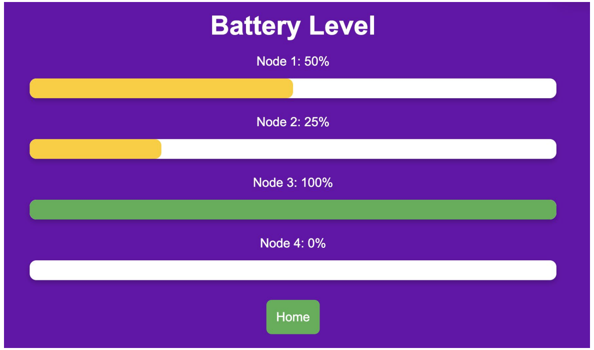
# Web interface

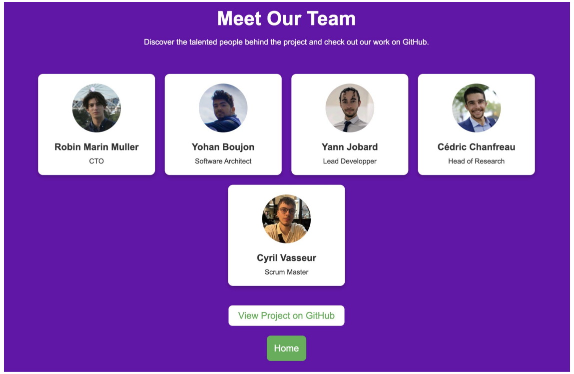




## Real time alert

## FFT visualisation




## Battery Level

## Team

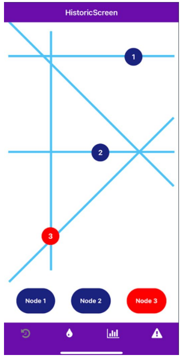
# Mobile application

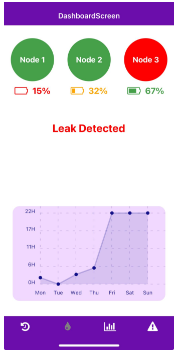
**Main Screens**:

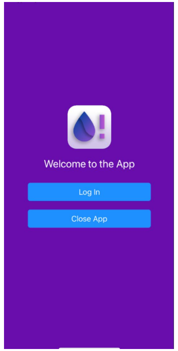
- **Home Screen:** User authentication (Log-in / Log-out)
- **Dashboard Screen:** Real-time node status (battery level, leak detection)
- **History Screen:** Node mapping over the network pipe
- **Statistics Screen:** Water consumption over time
- **Leak Screen:** Alerts and leak history

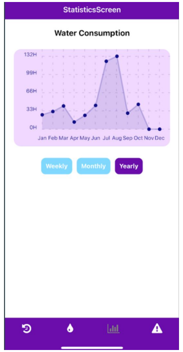
**Key Features**:

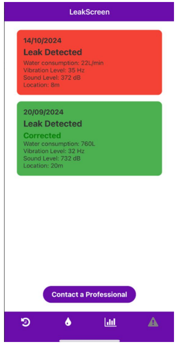
- **Cross-Platform** – Developed with **React Native & Expo** for Android and iOS
- **Intuitive Interface** – User-friendly dashboard for real-time monitoring and quick actions
- **Data Synchronization** – Integration with the server for live updates
- **Comprehensive Tracking** – Monitors leak history, node status, and water usage trends



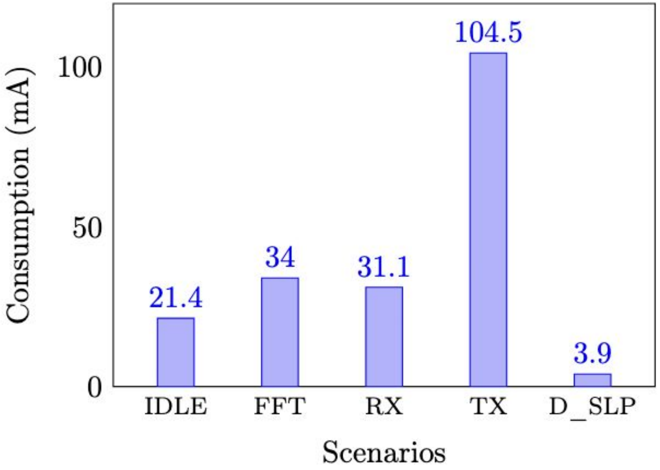



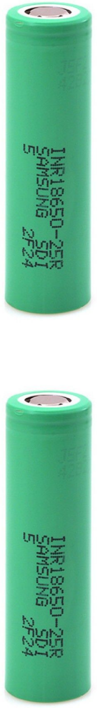



# Energy Management




**Energy Consumption of node V1**

**Real Application**



With two accu INR18650-25R :

Capacity for one : 2500mA/h
Node consumption : 3,9mA

5000/4 = 1250 h

almost 52 days of autonomy

# Results

Controlled tests confirmed accurate leak detection and signal classification.

Successful LoRa communication even in noisy environments.

Effective node to website/mobile application communication.

Succeeded using the Agile method.

Mark 1 fully working. Mark 2 in progress.

Great teamwork.

# Challenges and Future Work

Signal interferences in high-noise environments. → Refine AI models for better accuracy.

Untested prototype under real-world conditions. → Deploy the network in a scaled physical model prototype.

Current physical model prototype not as effective has we wanted. → Redesign/build *(asking help to a plumber).*

# Thank you for listening!

# Kick Starter